AI 创新实践 | 「架构师 Agent」系统化落地

原创 刘瑞洲 阿里技术 Aug 27, 2026, 3:06 AM 浙江 33 listened



这是2026年的第 55 篇文章

（ 本文阅读时间：约 20 分钟 ）

前言

当前，在业界，AI 仍然无法很好的理解“复杂技术系统”，也无法针对性的达成完善的技术方案设计，“架构师”仍然只能由“人”担任。繁忙业务的架构师每天仍然要参与 N 多个技术讨论、技术决策，去讲自己的技术思考、技术设计—— 架构师的人力瓶颈明显。

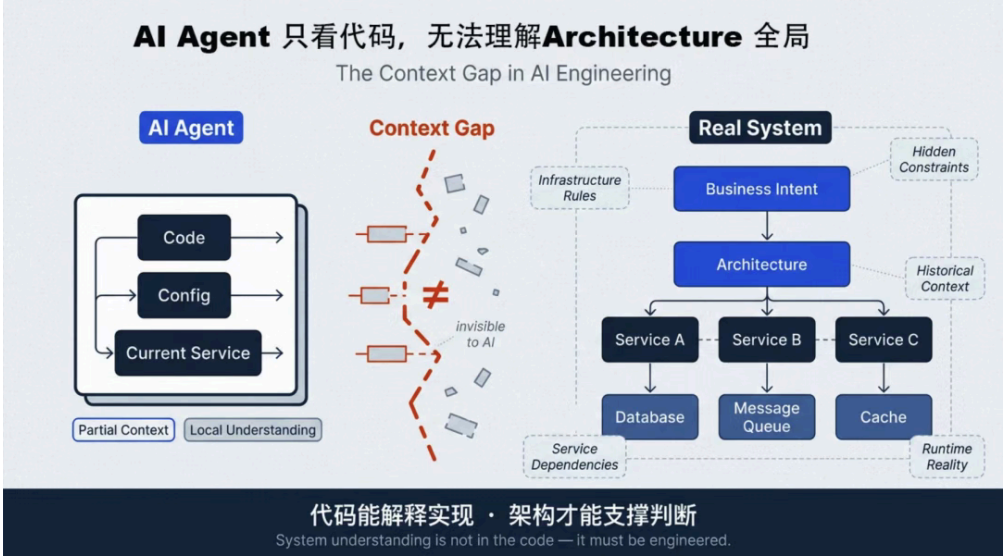
单个技术系统内部AI 逻辑理解和推理，在业界已经有了比较好的落地，但对于“跨复杂业务系统的架构理解与设计”并没有比较好的系统性解决方案提出来；而这个 topic 本身是迈向 7 x 24 小时Agentic Coding 的必经之路，AI 需要对复杂系统有很好的理解之后，才能针对性的做好技术方案设计、问题排查、乃至无人值守的自动化迭代。

我们希望，让大型存量分布式系统成为 AI 可理解、可推理、可验证的工程系统，让AI Agent可以真正的变成「架构师」。

01
AI 仍难以理解复杂系统架构

很多团队第一次使用 AI Coding 时，都会发现一个类似的问题：**从零做一个小系统，效果往往很惊艳；一旦把 AI 放进运行多年的大型系统里，效果就开始不稳定**。

小系统里，AI 能在很短时间内搭出页面、接口、数据模型、测试和部署脚本。大型复杂系统里，一个看上去不复杂的需求，可能涉及十几个仓库、几十个接口、若干配置中心和异步消息链路。AI 写代码本身未必慢，真正慢的是它不知道该从哪里开始，也不知道哪些地方绝不能动。



这不是一个单纯的模型能力问题。代码生成能力已经足够强，局部修改、补测试、解释调用链，很多模型都做得不错。困难在于，**复杂系统真正重要的知识并不完整地存在于代码中**。

一个字段为什么不能删除，某个历史分支为什么必须保留，一条 MQ 消息为什么只能新增字段不能改语义，某个接口为什么不能在网关层做校验，某个配置为什么必须跟发布灰度一起生效……这些信息可能散落在历史方案、事故复盘、配置平台、同事经验和没有被正式记录的约定中。人类工程师靠长期参与、沟通和事故记忆补齐了这部分上下文；AI 如果没有被提供这些事实，只能依据局部代码做推断。

所以，AI 在复杂存量系统中最容易出现的错误，往往不是代码语法错误，也不是编译器或单测能直接发现的错误，而是「局部正确、整体错误」：逻辑写在了不该承担职责的服务里；增加了一个看似方便的同步调用，却消耗了核心链路的超时预算；删掉了一段看似冗余的兼容逻辑，破坏了老版本客户端或下游离线任务。

以架构师的核心职能技术方案设计为例，技术方案设计决定了需求应该落在哪些系统、改动应放在哪一层、哪些能力可以复用、哪些约束不能突破，以及改完以后如何证明它仍然安全。AI Coding 的效率会放大方案正确性，也会放大方案错误的传播速度。方向错了，AI 只会更快地把错误落实成大量代码。

复杂存量分布式系统有一个很大的难题：AI 必须跨越业务语言、架构链路、多个服务和真实代码，才能获得足够上下文来生成技术方案。**很多团队目前的做法仍是由架构师或者资深工程师先完成人工方案，再把确定的改动点交给 AI 编码**；在与集团内外和多部门同事、同行的交流中，我们也观察到，**技术方案设计仍是人工介入最深的环节之一，仍然是需要架构师强力介入的环节**。

02
白纸与旧城：新系统容易，存量系统困难

除了系统规模以外，新系统、已有系统对于AI 的挑战也差别非常大。

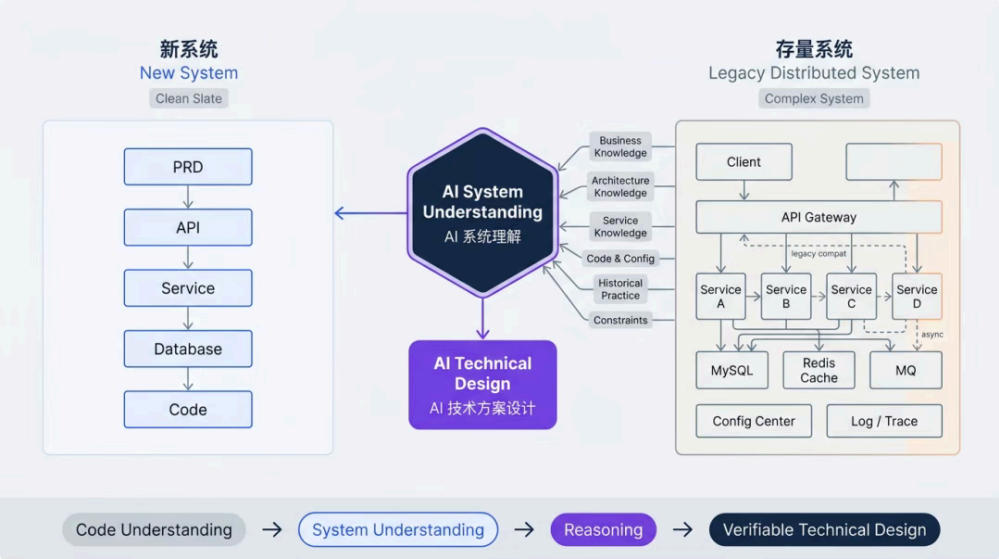
在AI 的加持下，一名研发工程师，就可以用一周的时间写出类似Salesforce 的系统，实现大部分基础功能。

但是如果让他在已有的Salesforce 系统上做一个中等规模的AI Coding 迭代，可能反而一周搞不定。

这是因为新系统、老系统，对AI 来讲差别也很大，核心差别在于历史上下文的认知、生效方式。

从零开始开发一个新系统，AI 面对的是一张相对干净的“白纸”。边界可以新建，模块可以新拆，接口可以新定，历史包袱也少。即使某些设计不够优雅，后续调整的成本通常仍然可控。

就相当于人一样，一个订单系统让一位新同学开发往往可以几天之内看到雏形，而如果让这位同学在已有的订单系统上开发，往往需要几天的时间去熟悉历史代码和历史业务场景。



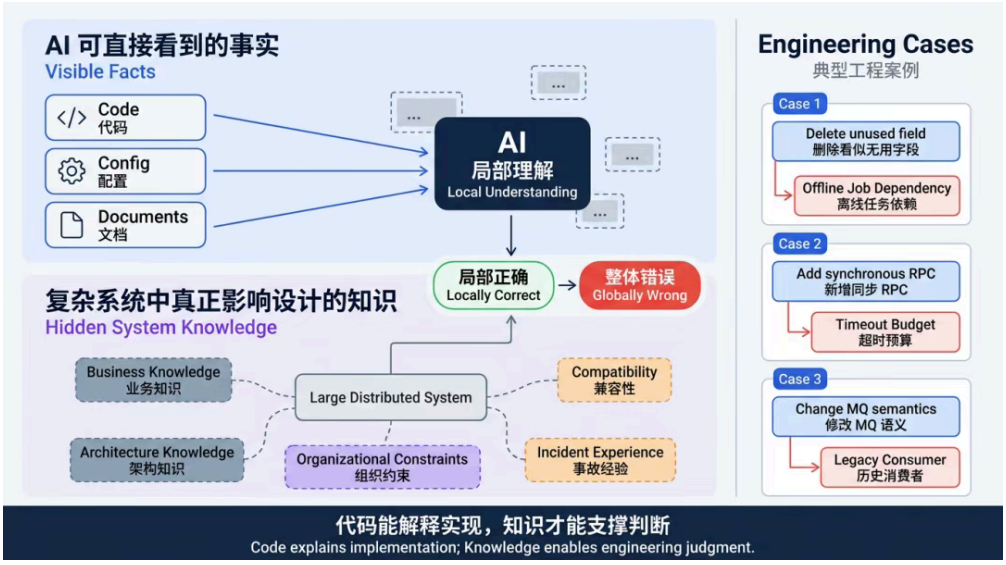
而存量技术系统像一座运行多年的城市。路网、管线、旧建筑、临时改造和居民习惯共同决定了它今天的样子。你不能只看一条街上的建筑风格，就决定把地下管线挖掉；也不能只看到一个服务里的代码，就断定它拥有某项业务的最终决策权。

大型分布式系统至少有四类知识同时影响方案设计。

- 第一类是业务知识。产品说“优化退款体验”“增加人群投放”“调整订单展示”，这些都是业务语言。它们对应的业务对象、状态含义、边界和例外，并不必然等同于代码中的类名和字段名。一个“订单”在不同团队的口语里，可能是交易订单、支付单、配送单或对账单；如果概念没有被消歧，后续检索和设计很容易从第一步就偏离。
- 第二类是架构知识。一个用户请求从客户端进入之后，经过哪些 BFF、领域服务、下游 RPC、消息和异步任务？哪些服务拥有数据主责，哪些只是聚合和展示？哪里允许最终一致，哪里必须强一致？这些决定了变更应该落在什么位置，也决定了影响分析的范围。
- 第三类是服务内部知识。单个服务中的 API 契约、领域对象、状态机、数据库、缓存、消息、定时任务、配置项和测试方式，决定了“具体怎么改才安全”。仅靠临时 grep，AI 可以找到一些相关文件，却未必知道哪个入口是主路径，哪个分支是历史兼容，哪个字段会被下游消费者依赖。

- 第四类是工程与组织知识。例如超时、重试、限流、灰度、发布、回滚、安全审批和监控规则。这些往往不改变业务代码的表面逻辑，却直接决定方案能否上线、故障是否可控。

从这个角度看，**“存量复杂系统”的困难在于知识彼此断裂**。代码只表达了部分实现事实；文档可能表达了一部分设计意图；配置记录了运行时行为；经验沉淀了大量没有写下来的约束。如果没有一套机制把它们组织起来，AI 就很难从一个 PRD 走到完整的技术方案。



这也是我们希望解决的痛点：让「存量的大型分布式系统」不再只存在于少数人的脑中，而是逐步成为 AI 可以全方位理解、进而参与方案设计、研发执行和线上排查的工程系统。

尤其「技术方案设计阶段」是整个AI Coding 的核心部分，是AI Coding 最重要的源头起点。

在公开可见的实践中，这种面向复杂存量分布式系统的端到端组合仍不常见，这部分能力目前业界一直没有很好的可靠解决方案。

足以证明达成这个目标确实是有难度、有挑战的，但是从价值来讲又是有必要性的，因此我们称之为 —— **难而正确的事**！

03
难而正确的事：行业走到了哪里

从 Spec、Harness 到系统理解

我们也调研了最近2年的相关技术设计、与探索实践，其实业界早就普遍意识到，直接把一句自然语言需求交给 Coding Agent，再期待它完成复杂研发，并不是稳定的工作方式，所以是持续有一些新的技术设计、技术方法论、实践产生：

- **第一条主流路线是 Spec-Driven Development**：GitHub Spec Kit 将流程组织为 `Spec → Plan → Tasks → Implement`，用结构化产物把需求、计划和任务串起来。Kiro 也将需求、设计、任务拆成独立文件；对于复杂、陌生或高风险的需求，它建议保留阶段性审阅和需求分析，而不是直接进入执行。

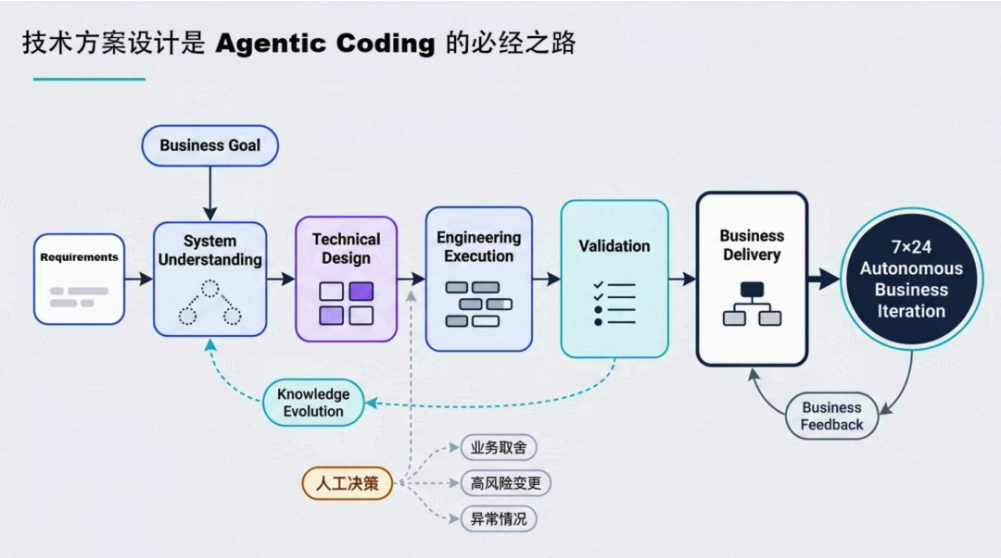
- **第二条路线是长程 Agent 的 Harness**：Anthropic 在长程开发实验中发现，只给模型一个高层目标并不足以得到生产级结果，需要通过初始化、任务拆分和结构化交接，让后续会话知道前面已经完成了什么、还缺什么。这解决的是长时间执行过程中的上下文连续性问题。
- **第三条路线是 Agent-friendly Repository**：OpenAI 的实践强调：给 Agent 一张地图，而不是一千页说明书。短小的 `AGENTS.md` 负责路由，结构化文档负责承载事实，CI 和自动化检查负责发现知识陈旧、链接失效和结构漂移。

这些思路和方向都很重要，也和我们的实践高度一致。不过这些实践比较多的还是偏单个微服务系统内部实践，对于跨复杂多系统的工业实践层面，并没有给出比较理想和细节的设计。

而我们希望向前一步，让AI 能够完成“复杂系统理解”，并能够真正做到类似于人类架构师一样实现跨多系统、甚至多终端的技术方案设计、推理等。

04
从技术方案设计Agent 开始

虽然我们要做的是「架构师Agent 」，但是为了避免空泛化的设计和过于发散的思考，我们把决定先从架构师的核心职能之一 —— 技术方案设计开始，逐步落地我们的架构师Agent。



图示：技术方案设计，是AI Coding、乃至 7 x 24 小时的 Agentic coding 的必经之路。

但是技术方案设计现在基本都是由架构师人工完成的，无法摆脱架构师的脑子和能力。

站在业界的角度，我们希望通过一种系统性的设计和落地，切实帮助到我们的日常工作，扩大AI提效的Scope 和高度。同时，也尽量形成可迁移的方法论或者流程，以帮助到更多的团队乃至业界的工程师。

我们的设计思路是**系统化地让 AI 完成技术方案设计：先理解需求，再理解业务和系统，再回源代码与配置，最后给出可执行、可验证、可审计的方案。这个过程的关键，不是一条更长的 Prompt，而是一套持续演进的知识工程与Agent 迭代路径。**

由此也可以看出，其中主要的几个抓手和关键点：需求质量、分层知识库（全面性、准确性、渐进式披露）、LLM基模（推理能力）、技术方案产出规范。

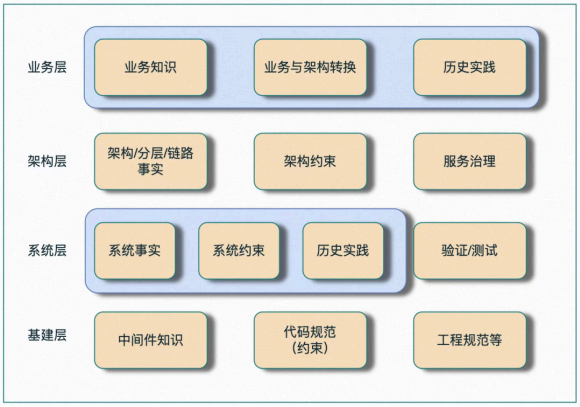
我们先从“知识库建设”开始第一步落地。

05
知识库的系统性建设落地

在之前的文章《分解一座冰山：后端系统“AI 知识库体系”建设实践》中已经系统性的讨论和设计过知识库体系，在文章发布后，也有很多不同部门的同学过来找我们讨论，包括大家各自在知识库建设过程中遇到的一些问题比如：指标衡量方式、什么样的知识库结构比较好、如何用指标衡量知识库建设效果等。

因此，**本文会根据我们之前已发表文章中的内容以及大家关心的话题，做更多进一步的细化阐述。**

尤其是AI 时代，对于知识库建设方面，很多团队天然想到的就是RAG。我倒不是不推荐RAG，**我们更认为知识库建设的第一层看到的不应该是RAG，应该是「领域」**。



注：引用上一篇文章的原图，其中蓝色背景框部分，是性价比比较高的模块，适合越早投入见效越快的模块。

5.1 知识库应该首先“对齐领域”：结构化的设计

1.5 年之前，我对AI 一直有一个比喻：“能力强大的小学生”。当时 AI 确实很强大，但是思路层面非常懵懂，经常需要复杂的提示词才能让他理解任务，并理解这个任务对应在现实世界中的实际逻辑，从而去更加准确的完成人类交给的任务。

而最近一年多的大模型迭代，让我们看到了非常不一样的能力，就是基模能力越来越强，**我现在已经很愿意把AI 基模形容为“能力强大的大学生”了**：对物理世界、人类需要的数字世界、各种商业逻辑、互联网业务逻辑等都有了比较强的内化性理解。

但无论是小学生还是大学生，我对团队同学提的建议是： **把AI 当做一名刚毕业的很聪明的实习生来引导，指导它的工作**。

虽然当前主流的大模型采用的 Transformer 架构和人脑神经思考反馈机制差异很大。但是从实践层面来看，其推理结果和原始输入是有非常大的强相关性的。原始输入越精确，得到的结论越精确、且越高效。这和人类的认知、思考模式的习惯也比较类似。

如果我们新入职的实习生同学，最快引导其工作的方式，应该也是给到他一个结构性比较强的文档，层层递进的熟悉和了解自己的工作，而不是丢给他一个文档库，让他自己去检索。

人类架构师对于技术系统的设计，是有很成熟的类似DDD（领域驱动设计）这样的方法论的。所以，我们的架构应该天然是有领域边界的，天然是“面向领域强结构化”的。

同样，为了提高AI 对于业务、系统的理解效率， 我也**强烈推荐让AI 理解这种“强结构化的领域知识”，而不是给AI 一个知识库让它自己搜**。

5.2 反直觉设计：为什么不首推 RAG

讨论知识库时，很多人的第一反应是 RAG：把历史 PRD、技术方案、会议纪要、接口文档、复盘和 Wiki 都放进向量库，需要时检索 Top K 片段交给模型。

RAG 很有用，但它不应该是复杂系统知识建设的第一选择。

原因很简单：检索解决的是「从资料中找出可能相关内容」，并不天然解决「AI 是否已经拿到了做出完整工程判断所需的知识集合」。

当数千篇非结构化文档进入同一个索引时，常见的问题会同时出现。

- **首先是颗粒度不一致**：一篇完整方案、一次会议纪要、一段接口说明、一个事故复盘里的截图说明，都可能成为检索单元。它们的知识密度差异很大。一个命中的片段看起来相关，并不代表它涵盖了这个问题真正重要的边界。
- **其次是语义相似不等于工程相关**：用户说“订单取消”，检索可能找到退款、履约、营销、风控中的大量相似描述，却不一定找到“状态机不可跳转”“某个 Topic 仍有历史消费者”“这个接口只有聚合服务可以调用”这类决定方案正确性的约束。
- **再次是 Top K 的完整性、结构性没有保证**：RAG 很容易返回若干看起来相关的文档，但这些文档可能来自不同时间、不同业务范围，甚至互相矛盾。模型拿到的是一些碎片，而不是一条完整链路。上下文越来越长，注意力却没有更集中，推理效率反而下降。

所以，RAG 的问题在于它很难独立承担知识建模的职责，把大量低结构、低密度、碎片化资料统一检索，得到的经常是一座更容易搜索、但依然杂乱的文档仓库。

这里对应的第一性原理思考就是：知识是有结构的，更适合被结构化的索引和理解。信息（比如新闻类内容）是结构化松散的，更适合被平行/平铺化检索。

5.3 “蒸馏架构师的大脑”：业务知识库的建设

在上一篇文章中，对于「业务知识库」结构，我们给出了如下这种结构设计。也有很多同学来找我们问如何实现这个知识库，效果如何等等。

```
business/
├── index.md
├── meta/
```

```
4  │   └── index.md
5  ├── principle/
6  │   ├── index.md
7  │   ├── timeout.md
8  │   ├── idempotency.md
9  │   ├── consistency.md
10 │   ├── degradation.md
11 │   └── compatibility.md
12 ├── scenario/
13 │   ├── index.md
14 │   └── scenario-*.md
15 ├── reference/
16 #关联
17 、引用内容
18 ├── practice/
19 │   ├── index.md
20 │   └── practice-*.md
21 └── history/
22     └── history-YYYYMMDD.md
```

其实**这层知识库的主要目标是“把架构师脑子里的隐性知识挖出来”，从而尽量摆脱“技术方案必须依赖架构师的隐形知识才能产出”的魔咒**，让AI 对人的依赖进一步降低，进而更快的迈向 7 x 24 小时的 Agentic Coding 和交付。所以，从这一点来讲，就是**把架构师大脑中的知识蒸馏出来，形成文档**。

而为了实现这个目标，我们开发了新的 skill `business-knowledge-distill`，这个 skill 是专门用来根据比如技术方案设计文档、技术稳定性体系梳理串讲文档等进行蒸馏，梳理成强结构化的 `business-knowledge` 的。

同时，因为很多文档中有“交互图”，这个skill 也会下载并分析对应的图片内容，一起转为结构化的业务知识库，从实际生成效果来看还是非常赞的，尤其在技术方案设计阶段发挥了很大作用。

以高德的实时公交业务为例，我们一次蒸馏后形成的业务知识库如下，文件内详细内容暂不展开。

```
1  .
2  ├── evidence
3  │   └── images
4  │       ├── b5bebae4a0aba2c0a551.png
5  │       ├── be6c21dc82612f040f2f.png
6  │       ├── d05045ed77ba108686e8.png
7  │       └── image-manifest.json
8  ├── history
9  │   └── history-20260817.md
10 ├── index.md
```

```
├── meta
│   ├── departure-timetable.md
│   ├── index.md
│   ├── line-and-station.md
│   ├── nearby-recommendation.md
│   ├── realtime-bus-info.md
│   └── service-roles.md
├── practice
│   ├── capacity-planning.md
│   ├── index.md
│   ├── scheduled-tasks.md
│   └── stability-risks.md
├── principle
│   ├── data-sync.md
│   ├── degradation.md
│   ├── index.md
│   └── traffic-governance.md
├── reference
│   ├── bus-data-server.md
│   ├── bus-platform.md
│   ├── bus-rs-nds.md
│   ├── crowd-engine.md
│   ├── gbfs-service.md
│   ├── index.md
│   ├── info-service.md
│   ├── opendi.md
│   ├── poi-search.md
│   └── srtbus.md
├── review-queue.yaml
├── scenario
│   ├── arrival-query.md
│   ├── bus-main-image.md
│   ├── data-sync.md
│   ├── departure-timetable.md
│   ├── index.md
│   ├── nearby-recommendation.md
│   └── search.md
└── source-manifest.yaml
```

这里再次详细介绍一下这个结构的主要意义：

业务知识库以领域为边界。每个知识库只服务一个稳定的业务领域，不把多个领域的内部规则混在一起，但是彼此可以通过 reference 进行引用关联。它至少包括五类内容：

- `meta`：业务元语、核心对象、别名、非同义词和边界；
- `principle`：跨场景复用的领域原则，例如幂等、一致性、超时、兼容和降级；

- `scenario`：从用户或业务场景映射到 API、服务、数据、消息、异常和补偿；
- `practice`：历史决策、事故教训、兼容原因和可复用模式；
- `reference`：与其他领域的关系和契约，不复制对方领域的完整知识。

这样的结构是为了让 AI 在处理需求时有确定的阅读路径。先读元语，避免概念混淆；再读场景，确定业务如何落到技术系统；遇到一致性、兼容或降级问题时，回到领域原则；需要理解历史原因时，再查实践。

换个角度看，固定结构提供的是“知识覆盖约束”。它告诉 AI：解决一个业务问题，至少要理解哪些方面；而 RAG 只能告诉 AI：这里有一些可能相关的资料。

这并不意味着放弃 RAG。RAG 很适合做长尾资料发现、历史文档定位、开放式问答和证据补充。当固定结构知识库缺少某个细节时，可以通过 KBase 等检索能力找到原始文档、历史方案或复盘，再把经过确认的结论回写到结构化知识中。

因此，更合适的关系是：固定结构知识库负责定义 AI 必须理解的核心事实；架构图谱负责服务寻址和链路分析；服务知识库负责约束和验证；RAG 负责发现和补证。前者是知识骨架，后者是扩展检索能力。骨架没有立起来，检索再快也很难让 AI 获得稳定的系统理解。

5.4 知识正确性需要维护机制，而不是一次性生成

知识库最大的风险不是缺少文档，而是文档过期后以“看起来很可信”的方式误导 AI。因此，知识维护必须成为研发流程的一部分。

服务知识库适合和 Git Push、Pull Request、版本发布建立联动。代码或配置发生变化时，Hook 或 CI 可以根据 Diff 识别可能受影响的 API、对象、数据库、消息、配置和测试知识，生成待更新的候选项，并校验目录结构、交叉链接、版本基线和必填字段。

但这里不能走向另一个极端：任何代码变更都自动覆盖知识。对 API 契约、数据库语义、MQ Schema、状态机、历史兼容和安全策略等高风险知识，自动化应负责发现变化、生成候选和阻止遗漏，人工仍负责确认语义。工具擅长守住“代码变了，知识不能完全不变”的底线；人负责判断“它到底意味着什么”。

对于刚刚提到的“业务知识库”同样需要两种互补机制。

- 第一种是日常增量维护：关键 API、关键业务链路、业务规则发生新增或变化时，关联业务文档、方案和实现事实进行蒸馏；有来源但尚未确认的内容标记为待审核，不能静默升级为领域事实。在稳定业务上，业务知识库的迭代频率是非常低的，增量更新可以用人工保障，也可以基于需求迭代release 做事件触发关联。
- 第二种是周期性校准。比如高德每年的五一、十一出行节**大促重点保障节点前后，会产出大量稳定性梳理、系统串讲、链路盘点和复盘资料。它们通常覆盖多个服务之间最容易遗漏的风险与经验，是校准业务知识库的高价值资料。**日常维护保证及时性，大促资料蒸馏帮助补齐跨系统、历史性和高风险知识，也是非常重要的触发时机。

知识的质量可以在迭代中持续考察或者评测效果，每条重要知识最好能够看到来源、最后确认时间、适用范围、当前状态和负责人；每份方案最好能看到关键结论来自哪里；每次代码变更最好能知道哪些知识可能需要同步更新。这样，知识库才不会变成另一座无人维护的文档墓地。

5.5 service-knowledge 到底有没有用?

在上一篇文章《分解一座冰山：后端系统“AI 知识库体系”建设实践》中我们提到了对于单个微服务系统的知识库设计和实现。

后续**有很多的同学过来找我们讨论 service-knowledge 的设计和思考**，集中的几个主要问题以及对应的思考、实践反馈，我在这里统一解答一下：

1. service-knowledge 是否一定要和代码放到一个 git repo?

a. 不一定，完全可以和代码分开；
b. 和代码在一起的优点是方便在技术方案设计、coding 、code review 等各个阶段的agent 能够快速摸清楚代码的结构和逻辑；
c. 但这并不是和代码维护到一起的主要原因， 仍然可以通过 agent runtime 或者 Harness 、SKILL 等实现本地文件目录中将二者进行关联；

2. service-knowledge 如何保证和代码的一致性?

a. 可以按照首次生成、增量生成来区分；
b. 首次生成问题不大，覆盖的全面性比较能保障；
c. 增量生成则可以关联 git hook，或者 coding agent hook 的方式来实现。coding agent hook 的方式我们在 产出物 `AGENTS.md` 文件中已经加入；而 git hook 则更适合远端异步维护的方式来生成 service-knowledge；

3. service-knowledge 和 LLM wiki (markdown) 的差异是什么?

a. 第一个大的区别是：这是一种基于DDD、边界、安全策略等业界成熟的方法论设计的知识库结构，基本可以屏蔽不同基模LLM 的能力差异，实现一致化的结构和知识—— 方法论是成熟的、跨项目是一致的、对AI 是友好的；
b. 第二个主要区别：YAML 格式的文件， 结构性更强，信息密度比markdown 等格式会更高一些，对大模型更加友好；

4. 既然代码即事实，service-knowledge 是否有存在的必要性？这个问题很多同学、同事问到了。

a. service-knowledge 的基础作用是 code indexing，可以非常高效快速的让大模型理解代码结构，而无需遍历所有代码，尤其当代码量超过5万行的时候，甚至差异会到一个数量级级别。对于这一点，使用 code graph 等插件也可以实现， 一般来讲至少提升 25% 的效率；
b. 第二重作用是抽取系统实体、已有logic 等：可以避免coding agent 每次都去分析和理解代码中的实体、逻辑等，可以大幅度降低这些重复工作，提高效率的同时也节省了 token 成本；
c. 多个微服务时更加AI Friendly：大家都知道大模型的 context window 是有限长度，而且前面的几十KB 占用了更高的基模 attention 。当一个 agent task 需要扫描10个微服务的时候，如果把这些微服务的代码都放入 context windows 则会瞬间触发记忆压缩，而压缩

则很容易导致记忆有损甚至丢失重要信息。因此，尤其在涉及到多个微服务的时候，service-knowledge 这种高信息密度、高结构化的知识的 AI Friendly 会更加明显；

5.6 渐进式披露：强结构化的知识路径

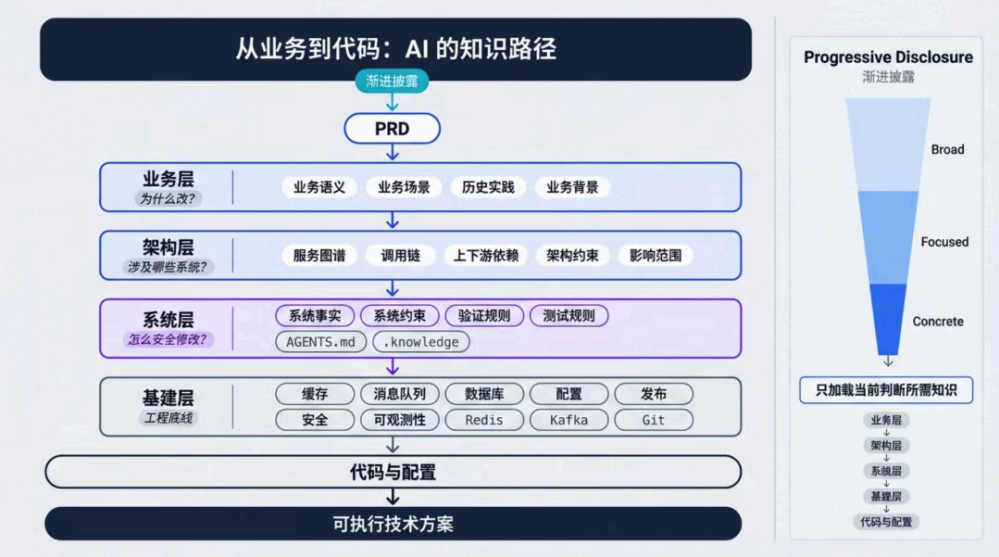
在知识准备完成后，**AI 不能把所有文档一次性读入上下文。大型系统的有效注意力仍然是稀缺资源。更可靠的方式是渐进披露：每一步只加载完成当前判断所必需的知识**，并让下一步由前一步的结论触发。

这条路径可以分成四层。

层级	AI 要回答的问题	主要知识来源	迭代频率
业务层	为什么改，业务落在哪里，业务元语在技术系统如何理解对应哪些系统或API	business-knowledge、产品资料、历史实践。 采用 `business-knowledge-distill` 蒸馏	低
架构层	涉及哪些系统，影响谁	aitom、服务图谱、接口和依赖关系	中
系统层	单个服务应该怎样安全修改	`AGENTS.md`、`.knowledge/`、代码和配置。 采用 `service-knowledge-generate` 生成	高
基建层	工程底线和上线规则是什么	中间件规范、发布流程、安全与稳定性规则。 静态内容，很少更新。	很低

这里主要是践行在上一篇文章《分解一座冰山：后端系统“AI 知识库体系”建设实践》中提出的分层思路。本文的重点在于把它用于技术方案设计：分层知识不只是用来“解释系统”，而是直接决定 AI 的调研顺序和设计边界，也契合“渐进式披露”的设计理念。

- 先从业务元语开始。需求中的“区域”“订单”“置顶”“灰度”等词，必须先被映射到确定的业务对象、状态和边界。没有这一步，后续在仓库和图谱中搜索时，AI 会把同名但不同义的概念混在一起。
- 再从业务场景进入技术链路。一个页面上的按钮、一个运营动作、一次用户状态变化，应该对应明确的入口 API、同步服务调用、异步消息、数据变化、异常处理和补偿逻辑。业务场景是 PRD 语言与技术系统之间的桥。
- 接着使用 aitom 等架构能力摸清上下游。它帮助 AI 回答：谁调用这个服务，它又调用谁，协议是什么，超时多久，有没有可以复用的能力，改动是否会跨越系统边界。对于跨服务技术方案，这一步比单仓库 grep 更重要。代码搜索可以告诉你“哪里出现过一个方法名”，图谱才能帮助你判断“它在系统里承担什么角色”。
- 最后进入目标仓库。`service-knowledge-generate` 生成的 `AGENTS.md` 和 `.knowledge/` 不应该是一份新的长篇 README。短小的入口文件负责告诉 Agent 如何路由；结构化知识负责描述系统边界、核心对象、API、下游依赖、基础设施、策略和测试要求。同时结合真实代码、配置和 Git 基线等核对当前事实，**并给出最终“代码级的技术方案设计”**。



这里有一个很重要的原则：不同事实应回到不同来源确认。生产系统当前行为以代码和配置为准；业务意图以已确认的产品和业务知识为准；历史原因以可追溯的实践记录为准。不能因为代码实现了某种行为，就把它自动视为未来需求的正确业务规则；也不能因为一份旧文档写过某种设计，就忽略现在代码已经发生的变化。

这种回源机制会主动发现「知识漂移」。比如在某一个需求“体验站三期”方案中，业务知识库曾记录某个模块“不排序”，而代码实际对所有条目统一按权重排序；方案据此修正了设计，并把知识库回写列入后续工作。更典型的是，`entrance` 仓最初因为业务关键词 grep 没有命中而被判断为无需改动，后续沿通用规则引擎继续追踪，才发现它承载了需要扩展的投放能力。一次方案版本修正说明了一个朴素事实：单一资料源给出的“结论”，在复杂系统里很容易不够可靠。

06
技术方案设计Agent 落地

在经历过上面的步骤，把业务知识库进行系统性建设之后，终于来到了最关键的环节，技术方案设计，这也是我们「技术方案设计Agent」最终的落地环节。

6.1 让 PRD 准入为技术设计的输入

技术方案设计不能从“产品已经写完 PRD”开始。因为很多 PRD 只是产品表达的起点，不一定已经具备进入研发的条件。

一个成熟的需求输入，至少要回答几个问题：用户是谁，在什么情境下遇到什么问题，现有替代方式有什么障碍，新的方案准备改变什么行为，最终用什么指标判断成功。缺少其中任意一环，AI 都可能把一个含糊的愿望翻译成一套貌似完整、实际跑偏的技术实现。

`prd-digest` 的价值在这里。它不负责替代架构师做详细技术设计，而是承担需求准入：检查问题与方案是否匹配，价值和影响是否有依据，范围是否受控，现有能力能否复用，验收与可逆性是否具备条件。对于材料不足的部分，它要明确标记“未知”或“待验证”，而不是替产品编造事实。

经过这一步之后，技术方案 Agent 接收到的就不再是一份原始 PRD，而是一份经过结构化整理的需求包：可验收目标、需求范围、明确不做项、关键假设、阻断项、待确认问题和专项评审触发项。

这一步看上去增加了流程，实际上减少了返工。AI 的速度会放大方向错误，先把需求逻辑链和边界讲清楚，才能避免它用十分钟写出一个需要两周返工的方案。

这里想强调的是：PRD 评审不是技术方案设计的前置行政动作，它是 AI 技术方案设计的第一个输入转换器。

6.2 新名词：产品需求增强

经过在集团内外走访，我们发现，在「产品需求准入」这一个环节，也是有两种常见的不同实践思路：

- 一种是产品需求准入机制，如上文提到；
- 另一种是“需求增强”机制—— 帮助产品需求PRD 文档变得更加健全，更有助于研发落地。

在没有AI agent 的年代，研发流程中并没有“产品需求增强”这一个环节，从务实的角度来看，是比较类似“架构师技术方案可行性分析”。

但是在AI 年代，我觉得“产品需求增强”是可以存在的，只是职责归属值得讨论，很多同学对此也有一些讨论和争议：

- **归属研发职能派：**有些团队会认为这个阶段是研发应该做的，因为产品需求增强核心是通过技术系统现状分析实现的，产品经理缺乏技术系统判断力，无法判断AI 做得增强效果是否靠谱。
- **归属产品经理职能派：**也有一些团队认为这个阶段的工作是应该产品经理实现的，因为需求增强本身涉及到“改需求”，而研发是没有权利单独改需求的。

其实这两种说法本身都有道理，如果真的让我评价的话，也并不能轻易说出谁更正确一些。但是我觉得更适合的解决方案是“架构师 Agent”来做，虽然我们的架构师Agent 是以“技术方案设计”为切入点来落地的。但实际上，具有技术方案设计能力的架构师Agent 是可以很容易拓展边界实现“产品需求增强”的职能的。让我们拭目以待。

6.3 Agent Context 与 Runtime

经过前面的系统化设计，我们分别完成了业务知识、架构与服务知识、系统分析工具以及技术方案设计 Skill 等能力建设。

但从 Agent 的运行视角来看，这些能力并不是彼此孤立存在的，整体分别作用于：Agent Context 和 Agent Runtime 调度——也就是Harness 阶段。

无论是命令行形式的 Agent Client，还是基于 GUI 的 Agent Client；无论是 MCP、Skill 等基础能力，还是 Agent Harness、Loop Engineering 等更高层的工程机制，本质上都是在持续丰富 Agent 的 Context 或者调度 Agent Runtime：为 Agent 提供完成复杂任务所需的**上下文、知识、工具、行为约束与运行机制**。

因此，前文的设计最终可以收敛为一个面向“跨多个复杂系统进行技术方案设计”的 Agent Runtime。

自顶向下，当前 Agent Runtime 主要包含以下内容：

a. 业务理解层：基于 KBase MCP 的业务支持库

通过 KBase MCP，为 Agent 提供结构化的业务知识访问能力，包括业务概念、领域模型、业务场景、设计原则、历史实践等。

它解决的是：

> **在进行技术方案设计之前，AI 是否已经理解了当前需求背后的业务语义与领域上下文。**

b. 系统分析与探索层：基于 AITOM 的 API 与链路追踪能力

通过 AITOM 提供 API 检索、调用关系分析、链路追踪等能力，使 Agent 可以从一个需求或业务入口出发，逐步探索真实系统中的相关系统、服务调用关系、上下游依赖、关键业务链路等；

c. 架构推理层：技术方案设计 Skill

技术方案设计 Skill 定义了 Agent 在复杂技术方案设计过程中的工作方式，包括如何理解和分析需求、如何逐步加载业务与系统上下文、如何定位相关系统与服务、如何对最终方案进行覆盖性与完整性验证等。

d. 服务知识层：多个微服务的 Service Knowledge

每个微服务沉淀自己的 Service Knowledge，用于描述服务级别的：业务职责与能力边界、核心领域概念、API 与事件契约、数据模型、关键依赖等。

e. 事实验证层：基于 Git Repository 的 Code Context

对于最终需要确认的实现事实，Agent 可以进一步访问对应微服务的 Git Repository，包括：代码实现、配置、API 定义、数据结构等。

6.4 AI 实现技术方案设计

基于上述设计，技术方案设计不再是一次性的长提示词生成，而是一套有输入、有检索路径、有验证和停止条件的推理过程。

在知识库完备、代码完备、PRD 准入完备之后，可以正式开始做技术方案分析和设计：

- 第一步，AI/Agent 读取经过需求准入的 PRD，提取可验收目标、范围、不做项、风险和待确认信息。
- 第二步，AI/Agent 在业务知识库中解析关键元语和场景。它要先确认“需求说的是什么”，再确认“这些业务概念在哪些系统、接口和数据上实现”。
- 第三步，AI/Agent 通过架构图谱定位候选服务和完整链路。这里要识别主路径、旁路、同步调用、异步事件、下游依赖、服务等级和架构红线，而不是只找代码关键词。
- 第四步，AI/Agent 进入每个候选仓库，根据任务类型读取相应的服务知识。例如新增 API 时，优先读取 API 契约、兼容策略、测试和政策约束；修改数据库时，优先读取表语义、迁移规则、下游依赖和验证要求。随后回源到当前 Git Commit、代码和配置。
- 第五步，AI/Agent 形成 Gap 分析。已有能力且语义一致的，明确复用；已有能力但需要扩展的，明确改造边界；代码和知识中都不存在的，明确新建。Gap 分析的重要性在于，它强迫方案回答“为什么这样做”，而不是直接罗列文件改动。
- 第六步，AI/Agent 生成完整方案，包括改动点、调用方影响、异常传播、兼容策略、测试矩阵、发布与配置动作、需求覆盖矩阵、知识库联动和待确认项。

Runtime 中的渐进式上下文加载：沿着最短认知路径完成任务

需要强调的是，所有的结构化知识并不会在一次任务开始时全部加载到 Agent Context 中。

对于复杂的跨系统技术方案设计，如果一次性将所有业务知识、服务知识、代码和工具结果全部注入 Context，不仅会造成大量无关信息干扰，也会增加 Agent 的上下文负担，降低推理质量。

因此，如“5.6节 渐进式披露”所述，本方案采用**渐进式披露（Progressive Disclosure）**的方式：

```
技术方案设计 Skill
        ↓
需求理解
        ↓
按需加载业务知识
```

```
        ↓
定位相关业务场景
        ↓
调用 AITOM 分析系统与调用链
        ↓
按需加载相关 Service Knowledge
        ↓
发现信息缺口
        ↓
回源 Git Repository 验证代码与配置
        ↓
形成技术方案
        ↓
关键结论进行证据验证
```

也就是说，Agent Runtime 并不是一个“预先加载所有知识的大 Prompt”。

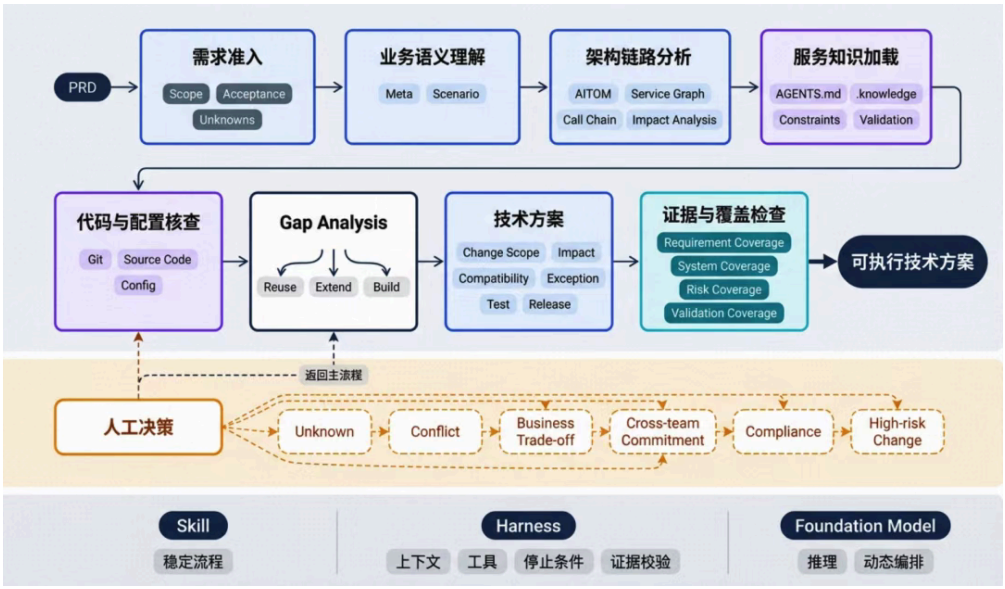
而是一个能够随着任务推进，持续完成：

> **发现问题 → 判断当前缺少什么 → 定位信息来源 → 按需加载 → 更新上下文 → 继续推理**

的动态运行环境。

从 Agent 的视角来看，前文建设的各项能力最终共同构成了其进行架构推理和技术方案设计的 Runtime。

它追求的不是让 Agent 知道所有信息，而是让 Agent 能够以尽可能短、尽可能准确的路径，获得完成当前架构决策所必需的上下文和证据。



这里的「纯 AI 做技术方案设计」，指的是上述调研、检索、链路推理、代码核查、文档生成和覆盖检查由 AI 独立完成，不再要求工程师先手工查完仓库、画完链路、写完方案再交给 AI。它并不等于取消人的职责。

当出现业务取舍、跨团队接口承诺、数据口径、合规要求或高风险变更授权时，AI 应停止猜测并发起明确的问题。人类介入的位置从“替 AI 收集所有资料”转为“裁决事实缺口和承担关键决策”。

这个变化很大：人不再是系统知识的唯一检索器，而是知识质量和决策权的最终负责人。

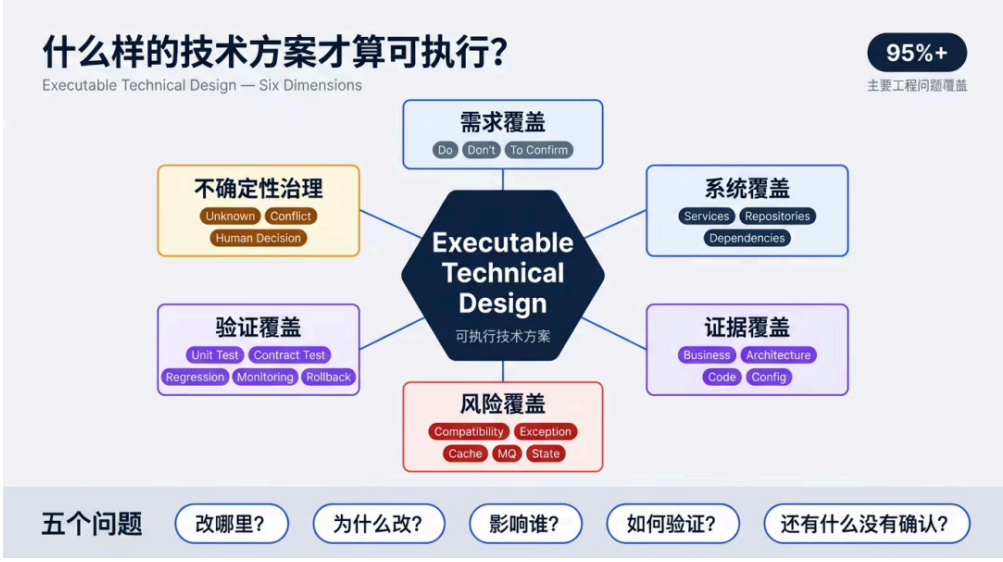
从 Harness 的角度看，Skill 负责固化稳定流程、输入输出和边界；Harness 负责组织上下文、工具调用、停止条件、证据校验和验证闭环；基础模型负责推理、规划和动态编排。三者各自承担不同职责。只靠模型，容易遗漏稳定约束；只靠流程，容易把模型能力锁死；只靠文档，则很难确保文档被正确加载和使用。

6.5 什么样的方案才算可执行

AI 写出一篇逻辑通顺的文章不难，AI 写出一份可执行的技术方案，要求高得多。

一份好的方案至少应包含以下内容。

- 首先，明确涉及仓库和范围外参与方。大型需求经常需要多个服务、客户端、运营后台、配置中心甚至外部平台共同完成。方案要说明哪些仓库必改，哪些系统只读不改，哪些工作虽然必要但不在本方案覆盖内。否则实施阶段很容易出现“每个人都以为别人会做”的空档。
- 其次，给出术语对照和代码现状。PRD 语言、业务语言、配置名、服务名和代码名经常不一致。把它们一一映射出来，既能避免搜索漏项，也能把业务概念转换为具体工程对象。
- 再次，使用“复用、改造、新建”组织 Gap。复用不等于看到类似代码就直接调用，必须确认用户、规则、状态、权限和风险语义是否一致；改造要说明扩展点和存量兼容方式；新建则要证明现有能力确实无法承载。
- 然后是影响分析。被改造函数有哪些调用方？异常会如何传播？哪些存量行为必须逐字节不变？修改一个字段、函数签名、缓存结构或配置项，可能影响的并不只是眼前接口。方案如果没有这部分，往往只是“改动清单”，还称不上技术设计。
- 验证部分同样不能只写“补充单测”。新增 API 要看契约测试，修改状态机要覆盖核心流程，调整消息要验证生产者和消费者兼容，修改缓存要关注失效和降级，涉及灰度和配置要有开关关闭时的存量行为回归。体验站三期方案中，除了单测，还给出了手工回归矩阵、异常传播链、需求覆盖矩阵和不改动约束；这些内容共同构成了方案的可执行性[8]。
- 最后，必须把未知信息留在文档里。待确认项不是方案不完整的证据，恰恰相反，无法通过现有资料证实的内容被明确登记，才说明方案没有把推断伪装成事实。



可以用五个问题检查一条需求是否真正被方案覆盖：改哪里？为什么改？影响谁？如何验证？还有什么没有确认？如果这五个问题都能回答，方案才有可能进入研发执行。

6.6 AI 设计技术方案完备度衡量

“技术方案完善度达到 95% 以上”，这是我们具体实践中，团队同学对于AI 生成技术方案的完成度定下的目标。

在这里，它不是指 AI 能够预知所有线上情况，也不是指任何需求都不需要人确认。它指的是：在需求已经完成准入、关键知识能够回源、代码与配置得到核查、人工评审完成之后，方案对主要工程问题的覆盖程度足够高。

这个指标可以拆成六个维度：

- 需求覆盖：PRD 条目是否逐项归属为“做、不做或待确认”；
- 系统覆盖：涉及服务、仓库、配置、上下游和范围外参与方是否完整；
- 证据覆盖：关键结论是否能回源到业务知识、架构事实、代码、配置或已确认文档；
- 风险覆盖：兼容、异常、灰度、缓存、消息、状态机和安全约束是否被检查；
- 验证覆盖：单测、契约、回归、监控、发布与回滚是否清楚；
- 不确定性治理：未知和冲突信息是否被显式登记，而没有被 AI 擅自补全。

在“体验站三期”需求的案例中，在原始需求完整度相对较高、且知识库体系完备的前提下，我们使用当前一流的基模，AI Agent 原生设计出来的技术方案覆盖度可以达到 95% 以上。

剩余的不确定性通常来自跨团队决策、外部系统行为、数据口径和产品取舍。这些本来就不应由模型单独决定。把它们压缩到少量明确的问题，而不是让它们以隐性风险的形式藏在方案里，已经是很大的进步。

至此，我们的「技术方案设计Agent 」成功落地。

知识库体系是“多场景 Agent”基建

虽然本文知识库体系的主要落地场景是服务于「技术方案设计」，但从实际作用价值的角度来看，知识库是全流程复用的。

比如：技术方案设计和线上问题排查看起来是两件事，但底层需要的能力却非常接近。

方案设计是从需求向未来推演：这个能力应该落在哪条链路上，修改之后会影响什么。线上排查则是从现象向过去回溯：用户看到的问题经过了哪些服务，哪个配置、缓存、消息或下游依赖可能异常。

两者都需要业务元语来理解问题究竟指什么，都需要场景映射来找到入口，都需要架构图谱来定位跨服务链路，也都需要服务知识来检查状态机、配置、缓存和降级逻辑。在我们的线上问题排查 Agent 实践中，同一套体系性知识正是其理解系统和缩小排查范围的基础。

这意味着知识库建设的投入并不只服务于「让 AI 写代码」。它会成为研发调研、技术方案、代码修改、线上排查、事故复盘和新人理解系统的共同底座。一次把关键知识显式化，可以在多个环节复用。

08
从「技术方案设计 Agent」到 「架构师 Agent」

我们虽然是从技术方案设计切入，并不是因为架构师的职责只有写技术方案，而是因为技术方案设计恰恰是架构师最核心、最能体现系统性判断的一项工作。

面对一个需求，架构师需要理解业务意图，定位跨系统链路，识别服务边界和历史约束，判断哪些能力可以复用、哪些地方需要改造，并最终给出一条可验证、可落地的技术路径。

这个过程的创新，并不是某个模型天然比别人更会设计，而是尝试把需求准入、业务隐性知识、跨服务图谱、服务内部约束、代码回源和质量验证，连接成一条可维护、可执行、可持续演进的技术方案生产线。

前面构建的业务知识、架构图谱、服务知识、渐进式上下文加载，以及代码回源和证据验证，本质上都不是为了让 AI 生成一份更漂亮的技术文档，而是在逐步补齐 AI 完成架构判断所需要的**知识、上下文、推理路径和验证能力**。

因此，技术方案设计 Agent 并不是终点，而是“架构师 Agent”的第一个具体落点。

当 AI 能够稳定完成这条链路后，同一套系统理解和技术判断能力还可以进一步扩展到需求增强、跨系统影响分析、架构评审、线上问题排查，以及对 Coding Agent 的任务规划和技术约束。

这些任务的输出不同，但底层依赖的是同一种核心能力：

> **理解复杂系统，并基于真实的业务、架构和工程事实完成判断。**

轻松扩展“架构师Agent ”的能力边界

在实际研发流程中，如果我们希望“架构师Agent”拥有排查线上问题的能力，那我们只需要给其赋予对应的 LogHouse MCP、Service Runtime MCP 等能能力即可—— 在当前的基础上做“架构师Agent”的能力边界扩展是很容易的事情。

从这个角度看，本文的价值不在于定义一个新的模型能力，而在于尝试验证一种**系统性的工程创新方法**：通过把原本分散在文档、代码、架构和工程师经验中的知识，以及架构师完成判断所经历的流程，逐步显性化、结构化，并组织成 AI 可以理解、推理和验证的工程系统。

技术方案设计，是「架构师 Agent」的起点；而把架构师的系统理解与技术判断能力逐步工程化、可执行化，才是这项实践真正希望探索的方向。

参考资料

[1] GitHub. Spec Kit Documentation.

https://github.com/github/spec-kit/blob/main/docs/index.md

[2] Kiro. Specs.

https://kiro.dev/docs/web/specs/

[3] Kiro. Spec Best Practices.

https://kiro.dev/docs/specs/best-practices/

[4] Anthropic. Effective harnesses for long-running agents.

https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents

[5] OpenAI. Harness engineering: leveraging Codex in an agent-first world.

https://openai.com/index/harness-engineering/

[6] 分解一座冰山：后端系统「AI 知识库体系」建设实践（长文干货）

[7] 后端架构 AI Friendly 的标准与路径：面向无人值守开发时代的系统重构

> 以下同学对本文亦有贡献，在此一并致谢：@李岩@王刚@陈方



欢迎留言一起参与讨论~

AI Native · 目录

上一篇
从 Coder 到 Designer ： 电商团队数据研发的 Harness Engineering 实践

下一篇
代码生成越来越快，软件交付真正的瓶颈转向了哪里？

阅读 4594

留言 3

写留言



武文龙 北京 9月18日

我们过去半年的实践和这个还思路挺一致的



高迪 北京 8月31日

知识库还是非常难的，特别是迭代了很多年的项目



杜若 北京 8月27日

共鸣~